

为什么 17 次随机询问能找出 01 串中 1 的个数？

CCPC 秦皇岛 2026 F：从乘积询问到随机筛选

这道题的交互形式很简洁，但信息有点“绕”。

评测机藏着一个长度为 n 的 01 序列：

$$a_1, a_2, \dots, a_n, \quad a_i \in \{0, 1\}$$

并且保证至少有一个 1。我们要找出其中 1 的总个数：

$$s = \sum_{i=1}^n a_i$$

一次询问可以选择一个下标集合 I ，评测机会返回：

$$\left(\sum_{i \in I} a_i \right) \left(\sum_{i \notin I} a_i \right)$$

也就是说，它不会告诉你集合里有多少个 1，而是告诉你：

集合内的 1 的个数，乘以集合外的 1 的个数。

最多只能询问 17 次。

本文是对 CCPC 秦皇岛 2026 F 的复盘与题解，按交互题写法组织；若你所在平台提供的是非交互版本，需要按实际输入输出格式调整。

这篇文章想说明一件事：这题不是要直接“还原序列”，而是要不断排除错误的总数候选。每次询问返回一个乘积，它会给所有可能的答案 S 加上一条约束；随机询问多做几次后，错误候选同时满足所有约束的概率会迅速下降。

一、一次询问到底提供了什么信息

假设某次询问选择了集合 I ，集合大小为：

$$k = |I|$$

真实答案是 s 。令这次询问中，集合 I 里真实的 1 的个数为：

$$X = \sum_{i \in I} a_i$$

这里的 X 只是一次固定询问下的计数记号，不是概率空间上的随机变量。后面分析随机询问时，我们会重新写成 $X(\omega)$ ，明确表示它依赖于随机结果 ω 。

因为总共有 s 个 1，集合外的 1 的个数就是：

$$s - X$$

所以评测机返回：

$$y = X(s - X)$$

程序不知道真实的 X ，也不知道真实的 s 。它只知道两件事：

1. 这次询问选了 k 个位置；
2. 评测机返回了 y 。

现在枚举一个候选答案 S 。我们要问：

如果总数真的是 S ，有没有可能出现这次返回值 y ？

如果总数是 S ，设这次询问集合中的 1 的个数为 x ，那么集合外的 1 的个数是 $S - x$ 。因此必须有：

$$y = x(S - x)$$

这就是候选答案 S 能解释一次询问的核心条件。

二、如何严格判断候选答案 S 是否可行

对于固定的 S 和 y ，方程：

$$x(S - x) = y$$

等价于：

$$x^2 - Sx + y = 0$$

这是一个关于 x 的二次方程。令判别式：

$$D = S^2 - 4y$$

如果候选答案 S 可以解释这次询问，那么必须满足：

1. $D \geq 0$;

2. D 是完全平方数;
3. $S + \sqrt{D}$ 或 $S - \sqrt{D}$ 至少有一个能被 2 整除;
4. 对应得到的整数根 x 是合法的。

合法性不能省。因为这次询问只选了 k 个位置，所以集合内最多只有 k 个 1:

$$0 \leq x \leq k$$

同时集合外有 $n - k$ 个位置，因此:

$$0 \leq S - x \leq n - k$$

合起来，合法的 x 必须落在区间:

$$\max(0, S - (n - k)) \leq x \leq \min(S, k)$$

只要二次方程的某个整数根落在这个区间内，候选答案 S 就能解释这次询问；否则它就应该被删掉。

这里还要检查两个根。因为:

$$x(S - x) = (S - x)x$$

如果 x 是一个解， $S - x$ 往往也是一个解。比如:

$$2(7 - 2) = 10, \quad 5(7 - 5) = 10$$

它们对应同一个返回值，但含义不同：一个是集合内有 2 个 1，另一个是集合内有 5 个 1。程序不能只看其中一个。

三、为什么会出现平方差

后面的概率分析会反复用到一个恒等式:

$$x(S - x) = \frac{S^2 - (S - 2x)^2}{4}$$

它不是技巧，而是来自更基本的公式:

$$ab = \frac{(a + b)^2 - (a - b)^2}{4}$$

因为:

$$(a+b)^2 - (a-b)^2 = 4ab$$

在本题里, 令:

$$a = x, \quad b = S - x$$

由于:

$$a + b = S$$

所以:

$$x(S-x) = \frac{S^2 - (x - (S-x))^2}{4} = \frac{S^2 - (2x-S)^2}{4}$$

也就是:

$$x(S-x) = \frac{S^2 - (S-2x)^2}{4}$$

这个公式的直观含义是:

当两个数的和固定时, 它们的乘积只取决于它们相差多远。

x 越接近 $\frac{S}{2}$, 乘积越大; x 越远离 $\frac{S}{2}$, 乘积越小。

四、把一次随机询问写成概率模型

下面分析随机询问排除错误候选的能力。为了让概率论表述严谨, 先固定真实序列, 只把“查询集合如何生成”看成随机的。

采用一个理想模型:

每个位置独立地以概率 $\frac{1}{2}$ 被放入查询集合 I 。

设真实序列中为 1 的位置集合为:

$$T = \{i : a_i = 1\}, \quad |T| = s$$

真正影响返回值的, 其实只有这些为 1 的位置是否被选中。把它们编号为 $1, 2, \dots, s$, 定义随机查询的样本空间:

$$\Omega_q = \{0, 1\}^s$$

一个样本点写成:

$$\omega = (\omega_1, \omega_2, \dots, \omega_s)$$

其中 $\omega_j = 1$ 表示第 j 个真实的 1 被选入查询集合, $\omega_j = 0$ 表示没有被选入。采用乘积概率测度:

$$\mathbb{P}(\{\omega\}) = 2^{-s}, \quad \omega \in \Omega_q$$

定义随机变量:

$$X(\omega) = \sum_{j=1}^s \omega_j$$

也就是说:

$$X : \Omega_q \rightarrow \{0, 1, \dots, s\}$$

它表示一次随机询问中, 真实的 1 被选中的个数。因此:

$$X \sim \text{Bin}\left(s, \frac{1}{2}\right)$$

评测机返回的值也是一个随机变量:

$$Y(\omega) = X(\omega)(s - X(\omega))$$

后面的参考实现也按这个模型生成询问: 每个位置独立地以概率 $\frac{1}{2}$ 被选入集合, 允许空集和全集出现。空集或全集的返回值一定是 0, 信息量较少, 但不会破坏候选判定逻辑。

五、错误候选什么时候会误通过

固定一个错误候选:

$$S \neq s$$

在随机结果 ω 下, 若暂时不考虑集合大小 k 带来的合法性检查, 错误候选 S 能通过这次询问, 当且仅当存在一个整数 x , 使得:

$$x(S-x) = X(\omega)(s-X(\omega))$$

为了把这个条件写清楚，定义代数坏集合：

$$\mathcal{B}_S = \{t \in \{0, 1, \dots, s\} : \exists x \in \{0, 1, \dots, S\}, x(S-x) = t(s-t)\}$$

这里的 t 表示真实答案为 s 时，一次随机询问中被选中的真实 1 的数量。

若暂时不考虑 k 带来的合法性检查，则有：

$$S \text{ 在 } \omega \text{ 下的代数通过} \iff X(\omega) \in \mathcal{B}_S$$

但程序的实际通过还要求判别式、整数根，以及

$$0 \leq x \leq k, \quad 0 \leq S-x \leq n-k$$

因此：

$$\text{程序实际通过} \implies X(\omega) \in \mathcal{B}_S$$

也就是说， $\mathcal{B}_S$ 给出的是一个上界；加入合法性检查后，通过概率只会更小。

于是：

$$\mathbb{P}(S \text{ 程序单次通过}) \leq \mathbb{P}(\{\omega \in \Omega_q : X(\omega) \in \mathcal{B}_S\})$$

又因为：

$$\mathbb{P}(\{\omega \in \Omega_q : X(\omega) = t\}) = \frac{\binom{s}{t}}{2^s}$$

所以：

$$\boxed{\mathbb{P}(S \text{ 程序单次通过}) \leq \sum_{t \in \mathcal{B}_S} \frac{\binom{s}{t}}{2^s}}$$

如果 17 次询问相互独立，并且每次使用同一类随机模型，那么对固定错误候选 S 有：

$$\mathbb{P}(S \text{ 程序通过全部17次}) \leq \left(\sum_{t \in \mathcal{B}_S} \frac{\binom{s}{t}}{2^s} \right)^{17}$$

这就是随机询问的核心价值：单次看起来信息有限，但多次独立叠加后，错误候选会越来越难伪装。

六、坏集合 $\mathcal{B}_S$ 为什么不大

从坏值条件出发：

$$x(S - x) = t(s - t)$$

用前面的平方差公式：

$$x(S - x) = \frac{S^2 - (S - 2x)^2}{4}$$

$$t(s - t) = \frac{s^2 - (s - 2t)^2}{4}$$

代入并整理：

$$S^2 - (S - 2x)^2 = s^2 - (s - 2t)^2$$

也就是：

$$(S - 2x)^2 - (s - 2t)^2 = S^2 - s^2$$

令：

$$A = S - 2x, \quad B = s - 2t$$

得到：

$$A^2 - B^2 = S^2 - s^2$$

继续分解：

$$(A - B)(A + B) = S^2 - s^2$$

这一步很关键。它说明每一个坏值 t ，本质上都对整数 $S^2 - s^2$ 的某种因子分解。

因此：

$$|\mathcal{B}_S| \leq 2\tau(|S^2 - s^2|)$$

其中 $\tau(m)$ 表示正整数 m 的正因子个数。这里写 $\leq$ ，是因为不是每个因子分解都能还原出合法的 x, t ；还需要满足奇偶性和范围约束。

更具体地说，枚举整数因子：

$$u = A - B, \quad v = A + B, \quad uv = S^2 - s^2$$

反过来：

$$A = \frac{u+v}{2}, \quad B = \frac{v-u}{2}$$

然后：

$$x = \frac{S-A}{2}, \quad t = \frac{s-B}{2}$$

只有当 A, B, x, t 都是整数，并且：

$$0 \leq x \leq S, \quad 0 \leq t \leq s$$

时，才得到真正的坏值 t 。

七、一个具体例子： $s = 5, S = 4$

真实答案是 $s = 5$ ，错误候选是 $S = 4$ 。

错误候选能通过一次询问，意味着存在整数 x, t ，满足：

$$x(4-x) = t(5-t)$$

其中：

$$0 \leq x \leq 4, \quad 0 \leq t \leq 5$$

也就是说，我们要求：

$$\mathcal{B}_4 = \{t \in \{0, 1, 2, 3, 4, 5\} : \exists x \in \{0, 1, 2, 3, 4\}, x(4-x) = t(5-t)\}$$

令:

$$A = 4 - 2x, \quad B = 5 - 2t$$

则:

$$A^2 - B^2 = 4^2 - 5^2 = -9$$

也就是:

$$(A - B)(A + B) = -9$$

枚举 -9 的整数因子对, 可以得到坏值。例如 $(A - B, A + B) = (-3, 3)$ 时,

$$A = \frac{-3 + 3}{2} = 0, \quad B = \frac{3 - (-3)}{2} = 3,$$

于是

$$t = \frac{5 - B}{2} = \frac{5 - 3}{2} = 1.$$

完整枚举后得到:

$$\mathcal{B}_4 = \{0, 1, 4, 5\}$$

所以:

$$\mathbb{P}(\{\omega \in \Omega_q : X(\omega) \in \mathcal{B}_4\}) = \frac{\binom{5}{0} + \binom{5}{1} + \binom{5}{4} + \binom{5}{5}}{2^5} = \frac{12}{32} = 37.5\%$$

这只是一次询问。如果独立重复 17 次, 即使用这个单次概率粗略估算, 也会变得非常小。

八、当 $S < s$ 时, 一个更直观的上界

这一节给出一个更粗但很好懂的上界。它只讨论:

$$S < s$$

如果:

$$t \in \mathcal{B}_S$$

那么存在 x , 使得:

$$x(S - x) = t(s - t)$$

但对任意 $0 \leq x \leq S$, 都有:

$$x(S - x) \leq \frac{S^2}{4}$$

于是:

$$t(s - t) \leq \frac{S^2}{4}$$

再用平方差:

$$4t(s - t) = s^2 - (s - 2t)^2$$

得到:

$$s^2 - (s - 2t)^2 \leq S^2$$

也就是:

$$(s - 2t)^2 \geq s^2 - S^2$$

由于 $S \leq s - 1$, 所以:

$$s^2 - S^2 \geq s^2 - (s - 1)^2 = 2s - 1$$

因此:

$$\mathcal{B}_S \subseteq \{t : (s - 2t)^2 \geq 2s - 1\}$$

写成随机变量形式, 就是:

$$\mathbb{P}(\{\omega \in \Omega_q : X(\omega) \in \mathcal{B}_S\}) \leq \mathbb{P}(\{\omega \in \Omega_q : (s - 2X(\omega))^2 \geq 2s - 1\})$$

定义:

$$Z(\omega) = s - 2X(\omega)$$

即:

$$Z : \Omega_q \rightarrow \mathbb{Z}$$

并定义事件:

$$E = \{\omega \in \Omega_q : Z(\omega)^2 \geq 2s - 1\}$$

那么:

$$\mathbb{P}(\{\omega \in \Omega_q : X(\omega) \in \mathcal{B}_S\}) \leq \mathbb{P}(E)$$

接下来只要证明:

$$\mathbb{P}(E) \leq \frac{1}{2}$$

即可得到一个简单的单次误判上界。

九、用期望证明 $\mathbb{P}(E) \leq \frac{1}{2}$

回忆 $Z(\omega) = s - 2X(\omega)$ 。它表示一次随机询问中，两侧真实 1 的数量差。错误候选 $S < s$ 若想通过，就会迫使这个差足够大；而随机询问通常不会让这个差太大。下面用 Rademacher 变量和二阶矩把这个直觉落实为 $\mathbb{P}(E) \leq \frac{1}{2}$ 。

先计算 Z 的二阶矩。

定义 Rademacher 随机变量:

$$\varepsilon_j(\omega) = 1 - 2\omega_j$$

也就是:

$$\varepsilon_j(\omega) = \begin{cases} 1, & \omega_j = 0 \\ -1, & \omega_j = 1 \end{cases}$$

于是 $\varepsilon_1, \dots, \varepsilon_s$ 相互独立，并且:

$$\mathbb{P}(\{\omega \in \Omega_q : \varepsilon_j(\omega) = 1\}) = \mathbb{P}(\{\omega \in \Omega_q : \varepsilon_j(\omega) = -1\}) = \frac{1}{2}$$

所以:

$$\mathbb{E}[\varepsilon_j] = 0, \quad \mathbb{E}[\varepsilon_j^2] = 1$$

这里 ε_j^2 是函数 $\omega \mapsto \varepsilon_j(\omega)^2$ 的简写。

因为:

$$Z(\omega) = s - 2X(\omega) = \sum_{j=1}^s (1 - 2\varepsilon_j(\omega)) = \sum_{j=1}^s \varepsilon_j(\omega)$$

所以:

$$Z(\omega)^2 = \sum_{j=1}^s \varepsilon_j(\omega)^2 + 2 \sum_{1 \leq i < j \leq s} \varepsilon_i(\omega) \varepsilon_j(\omega)$$

两边取期望:

$$\mathbb{E}[Z^2] = \sum_{j=1}^s \mathbb{E}[\varepsilon_j^2] + 2 \sum_{1 \leq i < j \leq s} \mathbb{E}[\varepsilon_i \varepsilon_j]$$

由于独立且均值为 0:

$$\mathbb{E}[\varepsilon_i \varepsilon_j] = \mathbb{E}[\varepsilon_i] \mathbb{E}[\varepsilon_j] = 0$$

因此:

$$\mathbb{E}[Z^2] = s$$

这里 $\mathbb{E}[Z^2]$ 表示 $\mathbb{E}[\omega \mapsto Z(\omega)^2]$ 。

s 为偶数

当 s 为偶数时, $Z(\omega) = s - 2X(\omega)$ 也是偶数, 所以 $Z(\omega)^2$ 是 4 的倍数。

而 $2s$ 也是 4 的倍数。因此只要:

$$Z(\omega)^2 \geq 2s - 1$$

就必然有:

$$Z(\omega)^2 \geq 2s$$

定义事件 E 的指示函数:

$$\mathbf{1}_E(\omega) = \begin{cases} 1, & \omega \in E \\ 0, & \omega \notin E \end{cases}$$

逐点有:

$$Z(\omega)^2 \geq 2s\mathbf{1}_E(\omega)$$

两边取期望:

$$\mathbb{E}[Z^2] \geq 2s\mathbb{E}[\mathbf{1}_E]$$

而:

$$\mathbb{E}[\mathbf{1}_E] = \mathbb{P}(E)$$

所以:

$$s = \mathbb{E}[Z^2] \geq 2s\mathbb{P}(E)$$

得到:

$$\mathbb{P}(E) \leq \frac{1}{2}$$

s 为奇数

当 s 为奇数时, $Z(\omega)$ 也是奇数, 因此总有:

$$Z(\omega)^2 \geq 1$$

如果 $\omega \in E$, 则:

$$Z(\omega)^2 \geq 2s - 1$$

如果 $\omega \notin E$, 至少仍有:

$$Z(\omega)^2 \geq 1$$

所以逐点有:

$$Z(\omega)^2 \geq (2s-1)\mathbf{1}_E(\omega) + \mathbf{1}_{E^c}(\omega)$$

取期望:

$$\mathbb{E}[Z^2] \geq (2s-1)\mathbb{P}(E) + \mathbb{P}(E^c)$$

又因为:

$$\mathbb{P}(E^c) = 1 - \mathbb{P}(E)$$

所以:

$$s = \mathbb{E}[Z^2] \geq 1 + (2s-2)\mathbb{P}(E)$$

当奇数 $s \geq 3$ 时:

$$\mathbb{P}(E) \leq \frac{s-1}{2s-2} = \frac{1}{2}$$

当 $s = 1$ 时, 错误候选 $S < s$ 不存在, 因为题目保证答案至少为 1。因此无需单独处理。

综上, 对于任意固定错误候选 $S < s$, 在独立随机查询模型下:

$$\boxed{\mathbb{P}(S \text{ 程序单次通过}) \leq \frac{1}{2}}$$

于是 17 次独立询问后:

$$\mathbb{P}(S \text{ 程序通过全部17次}) \leq 2^{-17}$$

需要注意的是, 2^{-17} 是对固定错误候选的粗上界。若直接对所有 $S < s$ 使用并集界, 会得到 $(s-1)2^{-17}$, 这个界在 n 很大时并不紧。

实际程序还会叠加判别式、整数根、集合大小等检查, 因此筛选能力通常远强于这个粗界。本文的概率分析主要用于解释随机询问为什么有效, 而不是追求最紧的全局失败概率估计。

由于最终输出的是最小存活候选, 我们主要关心的是 $S < s$ 的错误候选是否会存活。

只要所有小于真实答案 s 的候选都被排除，真实答案 s 就会成为第一个存活候选。至于某些 $S > s$ 是否仍然存活，并不会影响最终输出。

十、算法流程

算法可以概括为：

1. 生成 17 次随机询问样本；其中空集或全集的返回值已知为 0，可以不实际向评测机发问，因此真实交互次数不超过 17。记录每次的集合大小 k 与返回值 y 。也就是说，数学上仍然把它看作一次随机询问，只是实现时省掉了向评测机输出这一步；
2. 从小到大枚举候选 $S = 1, 2, \dots, n$ ；
3. 对每个 S ，检查它是否能解释全部询问；
4. 输出第一个仍能解释所有询问的 S 。

注意：程序输出的不是“最后剩下的唯一候选”，而是最小存活候选。某些 $S > s$ 仍可能与全部返回值兼容，但不会影响答案。

判定函数的核心是：

$$D = S^2 - 4y$$

检查 D 是否为非负完全平方数，再检查两个可能的根：

$$x = \frac{S + \sqrt{D}}{2}, \quad x = \frac{S - \sqrt{D}}{2}$$

是否落在：

$$\max(0, S - (n - k)) \leq x \leq \min(S, k)$$

复杂度

随机样本数固定为 17，真实向评测机发起的交互询问次数不超过 17。完成全部询问后，从小到大检查每个 $S \in \{1, 2, \dots, n\}$ ，每个候选至多验证 17 次询问，因此判定部分的时间复杂度为 $O(17n) = O(n)$ 。

空间上，程序保存 17 组 (k, y) ，额外空间为 $O(17) = O(1)$ ；每次生成并输出询问集合时，临时数组最坏为 $O(n)$ 。

十一、参考实现

下面给出一份随机化参考实现。若随机询问没有充分区分真实答案和较小错误候选，程序仍存在失败概率；本文的概率分析解释的是筛选机制为何有效，而不是对全局正确率的完备保证。

代码按交互题写法给出；若平台提供的是非交互版本，需要按实际输入输出格式调整。

交互题实现还要注意三件事：

1. 每次输出询问后必须刷新缓冲区；
2. 如果评测机返回非法值，要立即退出；
3. 随机询问之间尽量独立；实现上可用 `uniform_int_distribution` 对每个位置独立抛硬币。

若随机到空集或全集，代码不向评测机输出该询问，只在本地记录返回值为 0；因此不会违反交互格式。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

bool check_one_query(int n, int S, int k, ll y) {
    ll D = 1LL * S * S - 4LL * y;
    if (D < 0) return false;

    ll r = sqrtl((long double)D);
    while (r * r < D) r++;
    while (r * r > D) r--;

    if (r * r != D) return false;

    auto valid_x = [&](ll x) -> bool {
        if (x < 0 || x > S) return false;
        if (x > k) return false;
        if (S - x > n - k) return false;
        return x * (S - x) == y;
    };

    if ((S + r) % 2 == 0) {
        ll x = (S + r) / 2;
        if (valid_x(x)) return true;
    }

    if ((S - r) % 2 == 0) {
        ll x = (S - r) / 2;
        if (valid_x(x)) return true;
    }
}
```



```

    }

    return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n == 1) {
        cout << "! 1" << endl;
        return 0;
    }

    mt19937 rng(
        chrono::steady_clock::now().time_since_epoch().count()
    );
    uniform_int_distribution<int> bit(0, 1);

    vector<pair<int, ll>> queries;
    queries.reserve(17);

    for (int t = 0; t < 17; t++) {
        vector<int> idx;

        for (int i = 1; i <= n; i++) {
            if (bit(rng)) {
                idx.push_back(i);
            }
        }
        // 空集或全集的返回值必然为 0; 不必真正询问, 但仍计作一次随机样本并记录约束。
        if (idx.empty() || (int)idx.size() == n) {
            queries.push_back({(int)idx.size(), 0});
            continue;
        }
        cout << "? " << idx.size();
        for (int x : idx) {
            cout << ' ' << x;
        }
        cout << endl;

        ll y;
        if (!(cin >> y) || y < 0) return 0;
    }
}

```

```

        queries.push_back({(int)idx.size(), y});
    }

    int answer = -1;

    for (int S = 1; S <= n; S++) {
        bool ok = true;

        for (auto [k, y] : queries) {
            if (!check_one_query(n, S, k, y)) {
                ok = false;
                break;
            }
        }

        if (ok) {
            answer = S;
            break;
        }
    }

    if (answer == -1) return 0;
    cout << "! " << answer << endl;

    return 0;
}

```

十二、小结

这道题的主线可以浓缩成一句话：

不去恢复整个 01 序列，而是用每次询问返回的乘积，反复筛掉不可能的总数候选。

从结构上看，它把四步接在一起：

乘积询问 → 二次方程 → 候选筛选 → 随机化降错

题面是交互题，核心却不是交互技巧，而是把评测机返回的乘积转化成可验证的代数约束，再用随机询问压缩错误候选的生存空间。概率部分说明随机询问为何有筛选能力；算法部分则通过最小存活候选，把这一能力落实为可运行的程序。

十三、几个容易写错的点

最后补充几个实现和理解时容易踩坑的地方。

第一，不能只检查一个根。

因为：

$$x(S-x) = (S-x)x$$

所以同一个返回值可能对应两个不同的集合内计数，程序必须同时检查：

$$x = \frac{S + \sqrt{D}}{2}$$

和：

$$x = \frac{S - \sqrt{D}}{2}$$

只要其中一个根合法，候选 S 就不能被删除。

第二，不能只看方程有没有整数根。

即使：

$$x(S-x) = y$$

有整数解，也必须检查 x 是否真的可能出现在这次询问中。合法区间是：

$$\max(0, S - (n - k)) \leq x \leq \min(S, k)$$

这里的 k 是询问集合大小，这个条件来自两个事实：

$$0 \leq x \leq k$$

以及：

$$0 \leq S - x \leq n - k$$

第三，概率分析里的 X 必须看成随机变量。

在固定一次询问时， X 只是一个计数；但在随机询问模型下，应该写成：

$$X(\omega)$$

它是定义在样本空间 Ω_q 上的函数:

$$X : \Omega_q \rightarrow \{0, 1, \dots, s\}$$

这样写可以避免把“某次询问的具体数值”和“随机询问的随机变量”混合在一起。

第四，本文的概率上界是在独立随机查询模型下得到的。

也就是说，我们假设每个位置独立地以概率 $\frac{1}{2}$ 被选入询问集合。

参考实现也采用这个分布，允许空集和全集出现。因此在概率分析中使用

$$X \sim \text{Bin}\left(s, \frac{1}{2}\right)$$

和代码是匹配的。空集或全集只会产生返回值 0，信息量较少，但仍然是这个独立随机模型的一部分。

十四、结语

如果只看题面，这道题很容易被理解成一道“如何设计交互策略”的题目。

但真正关键的地方在于：

评测机返回的不是一个随便的数，而是 $x(s - x)$ 这种带有强代数结构的乘积。

一旦把候选答案记为 S ，它就必须满足：

$$x(S - x) = y$$

这一步把交互返回值变成了二次方程，再通过判别式和合法区间，就能严格判断一个候选答案是否还能存活。

随机询问的作用，则是让错误候选越来越难同时解释所有返回值。

所以这道题最漂亮的地方，是它把三个东西接在了一起：

交互返回值 + 二次方程判定 + 随机化筛选

这也是为什么我们不需要知道每个位置到底是 0 还是 1，却仍然能够找到 1 的总数。